ADVENTIST UNIVERSITY OF CENTRAL AFRICA

Execution Evidence

Complete screenshot appendix

Distributed Multi-Model Analytics for E-commerce Data

Student: Florence Imanishimwe | ID: 101399

Submission: 11 November 2026

All 46 screenshots from the supplied evidence document are retained in their original order, across 23 source pages.

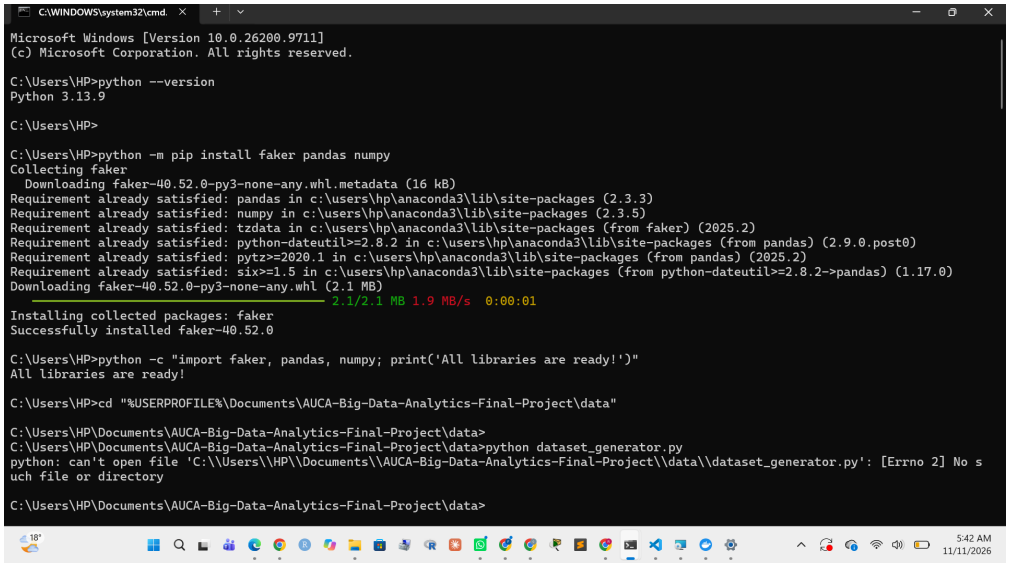
EVIDENCE GUIDE (appendix PDF page numbers)

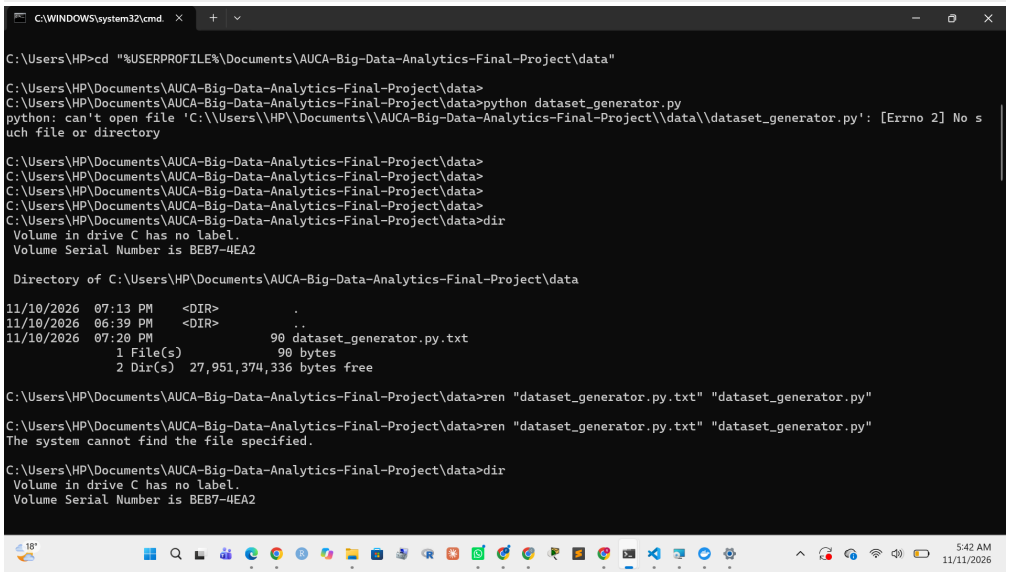
Environment setup and dataset generation	2-3
MongoDB import and analytical outputs	3-4, 19
Spark setup, SQL and funnel analysis	5-8, 23
Docker and HBase setup, loading and scans	8-17
Visualization generation and verification	18-19
GitHub version-control history	20-23
Product affinity analysis	24

Supplementary execution record; separate from the required 8-page technical report.

01 Python environment and initial setup

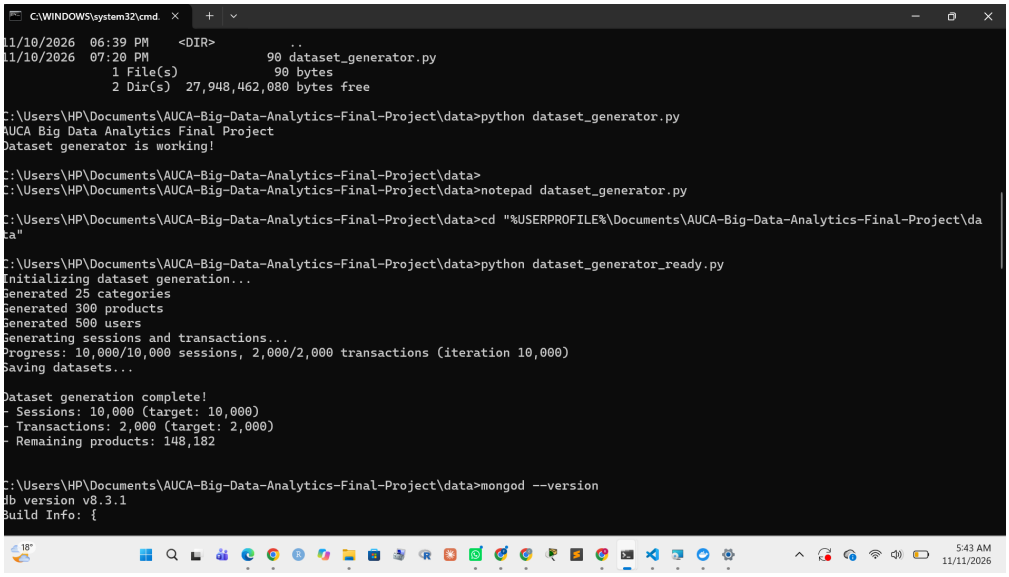
Python and required libraries were checked and installed. An initial dataset-generator file path was incorrect; the following steps show the correction.

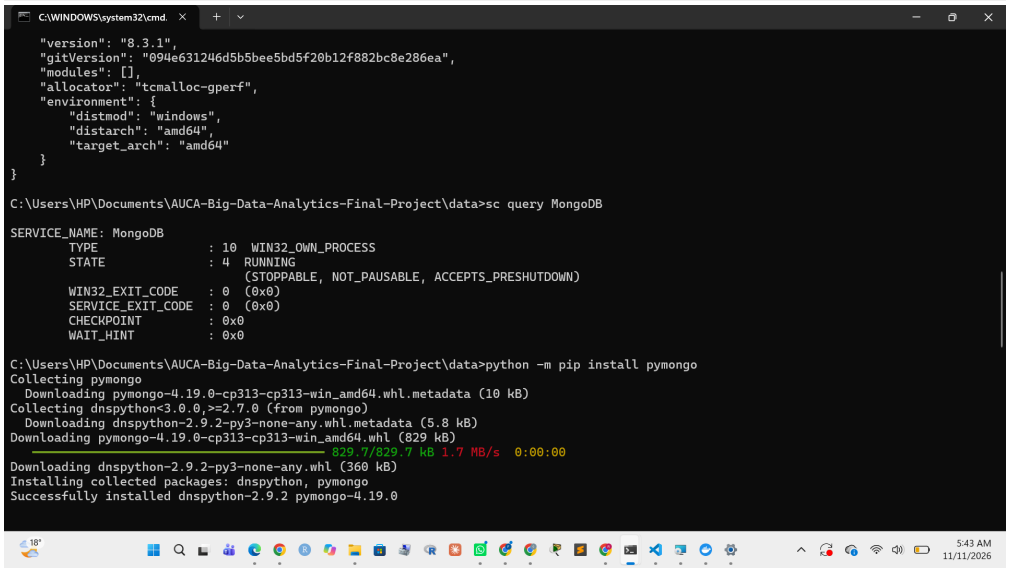




02 Dataset generation and MongoDB service

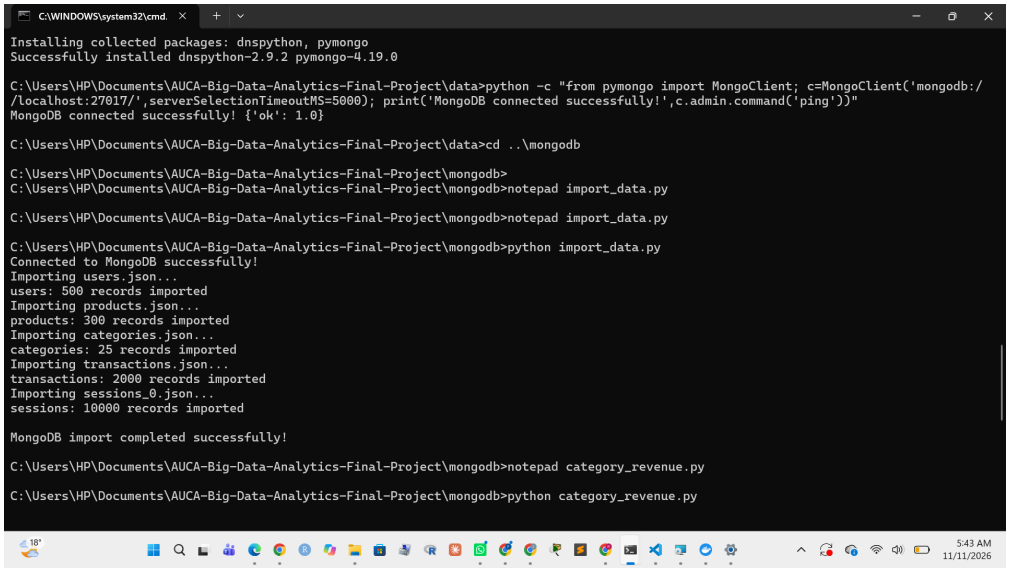
The corrected generator produced 10,000 sessions, 2,000 transactions, 500 users, 300 products and 25 categories. MongoDB service and Python dependencies were checked.

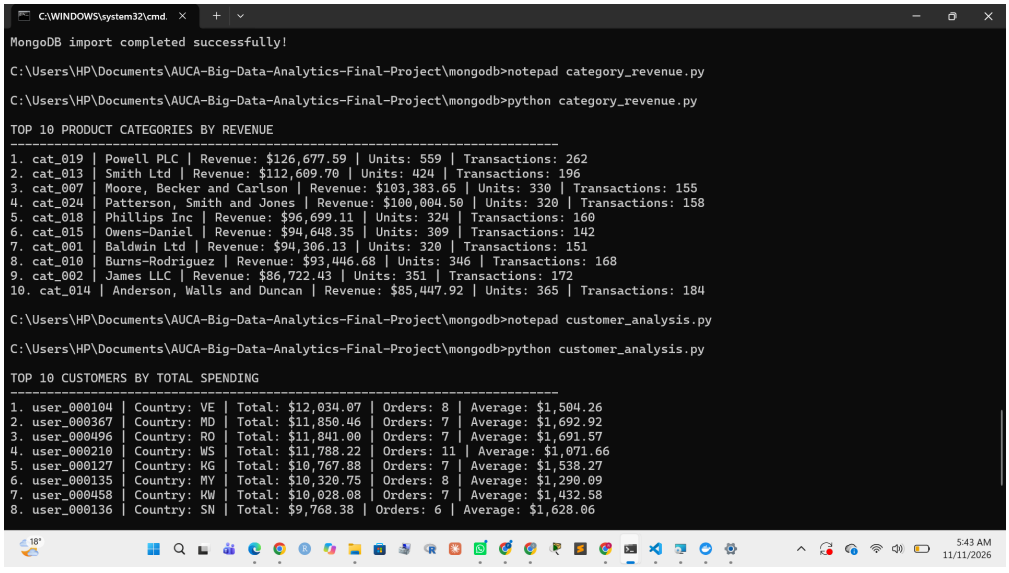




03 MongoDB import and analytical queries

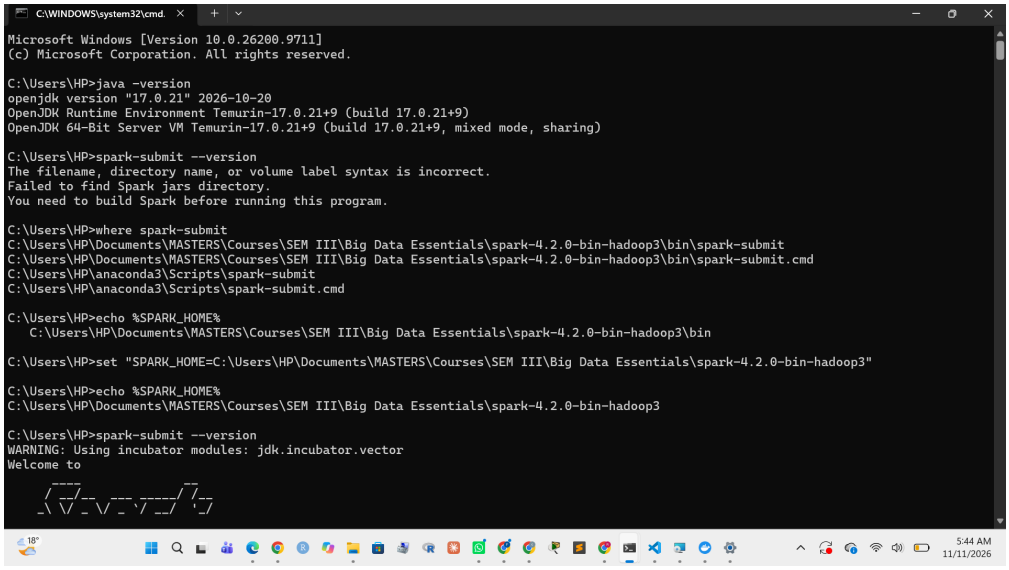
The data import completed successfully. Category revenue and customer-spending queries produced ranked results, including leading category cat_019.

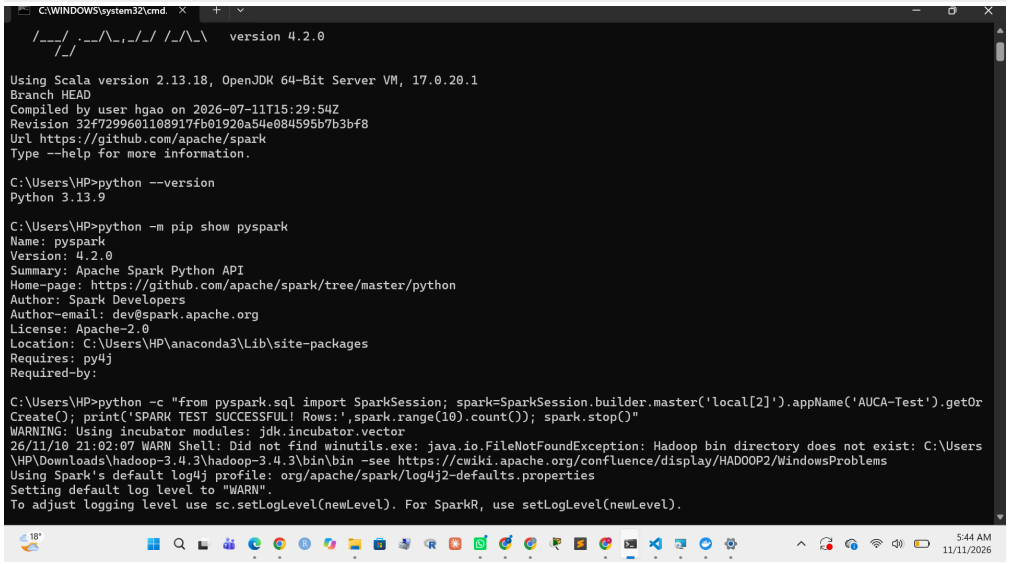




04 Spark and Java configuration

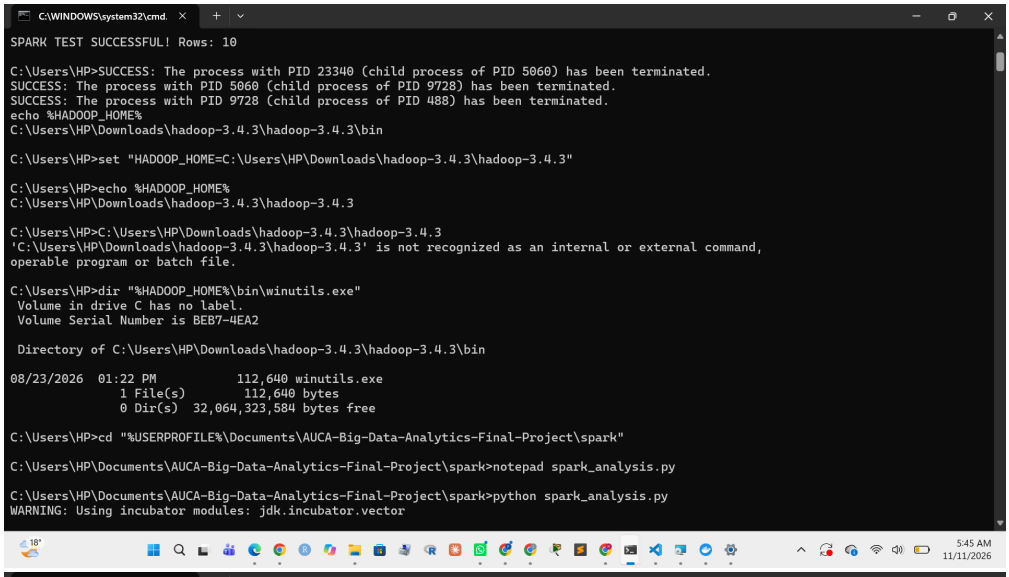
Java, Spark, and PySpark were checked. Initial environment configuration problems were addressed before Spark execution.

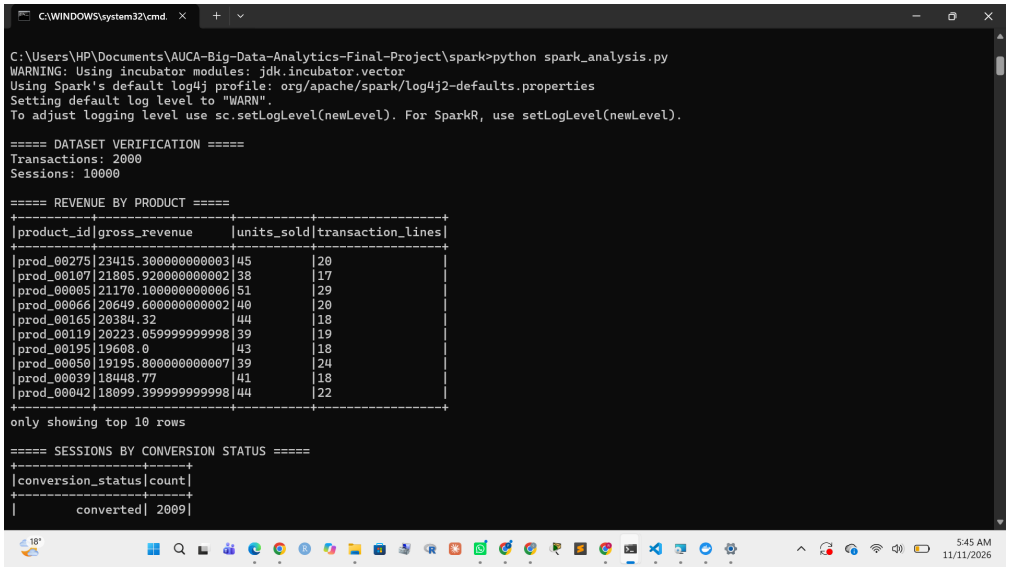




05 Spark batch analytics

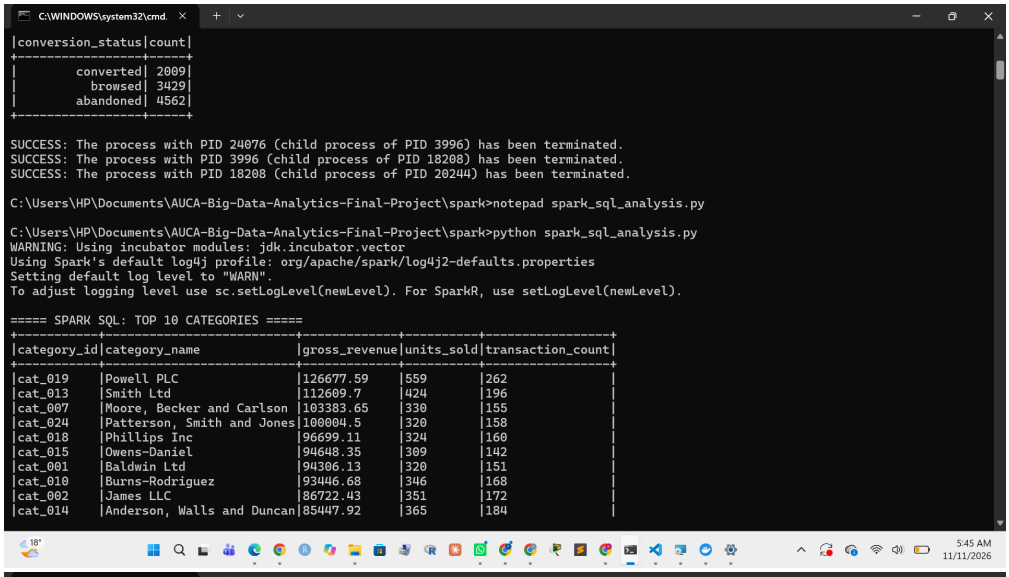
The Spark analysis ran and verified 2,000 transactions and 10,000 sessions, with revenue by product and session status summaries.

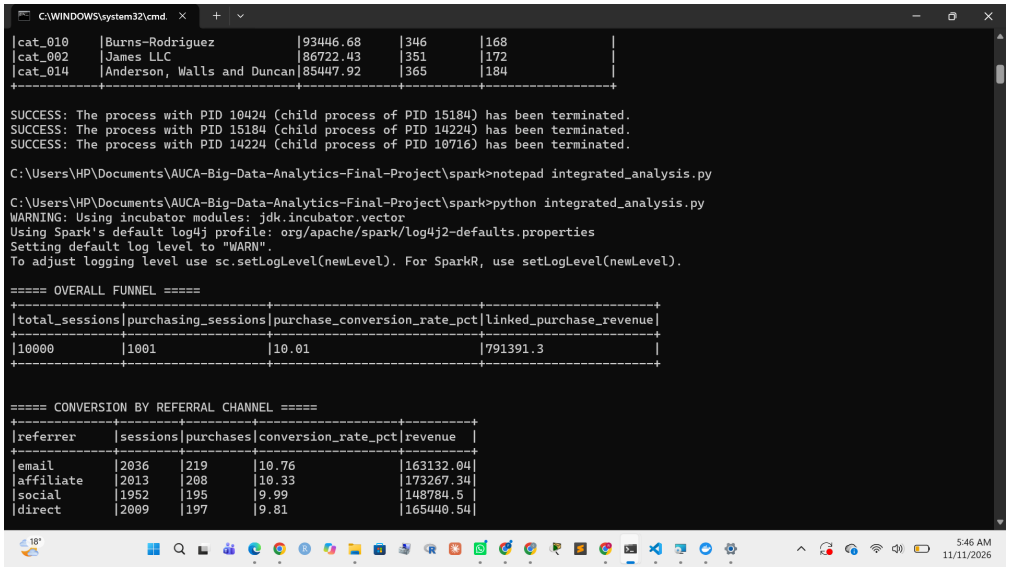




06 Spark SQL and integrated conversion analysis

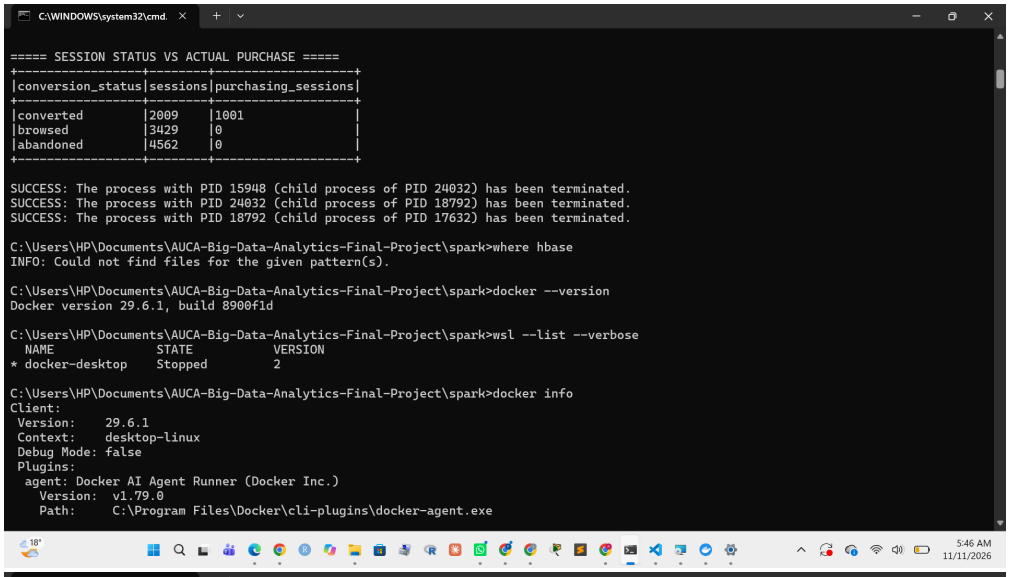
Spark SQL returned ranked categories. Integrated analytics linked 1,001 purchasing sessions to 10,000 sessions, yielding 10.01% purchase conversion.

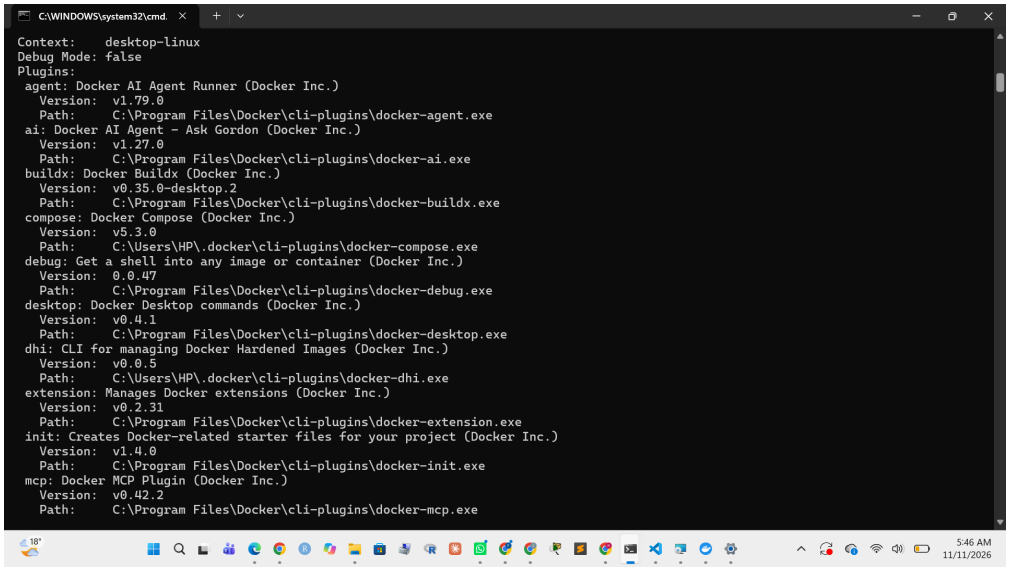




07 Conversion-status comparison and Docker checks

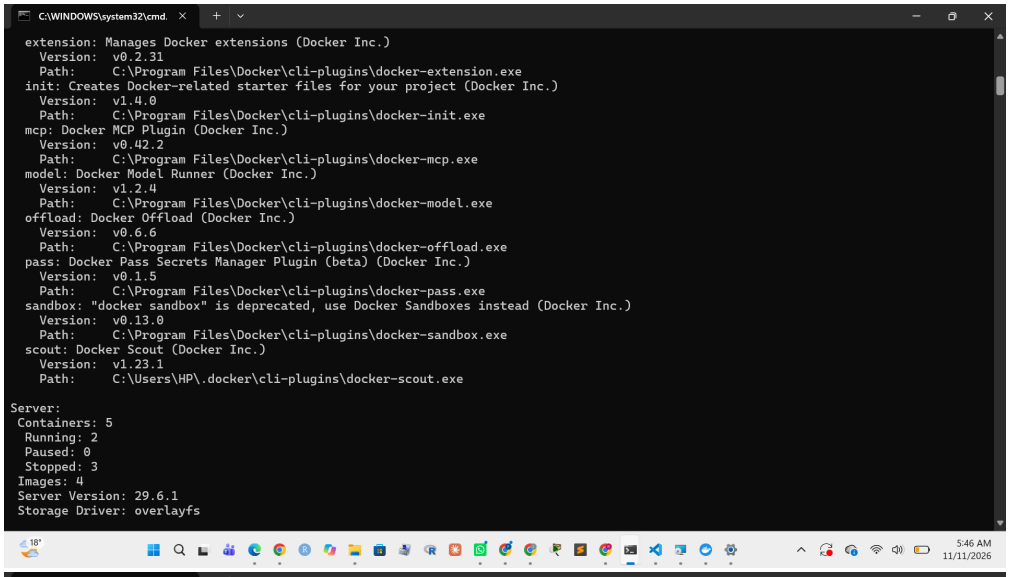
Session labels were compared with actual transaction-linked purchases. Docker availability was checked in preparation for HBase.

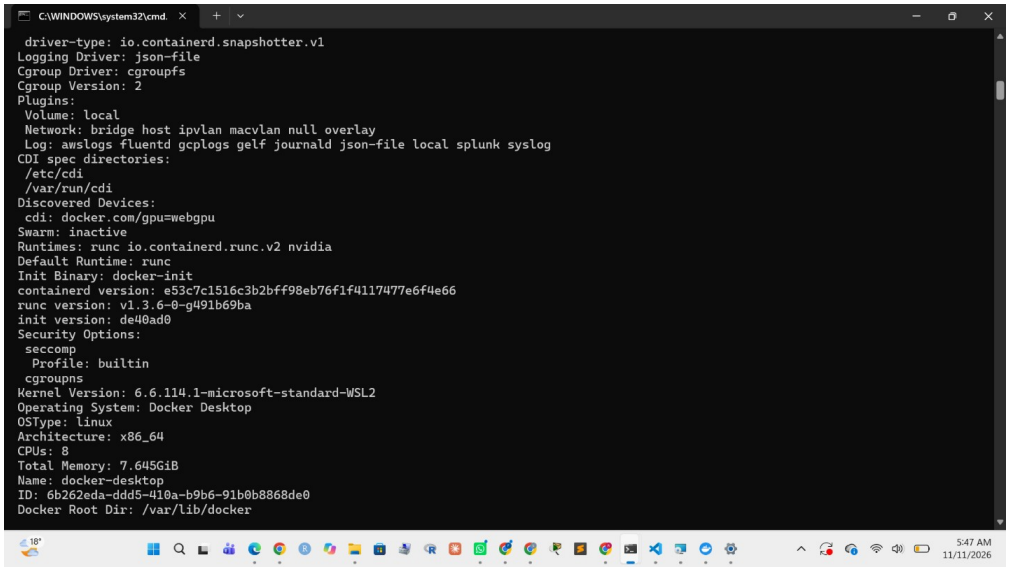




08 Docker environment details

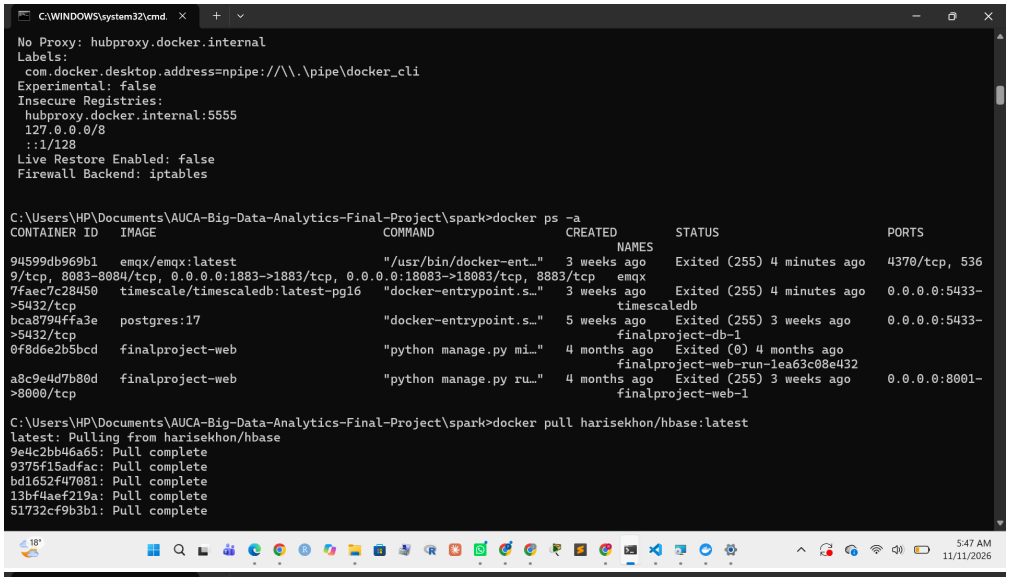
Docker client, server and plugin information was inspected to confirm the container runtime environment.

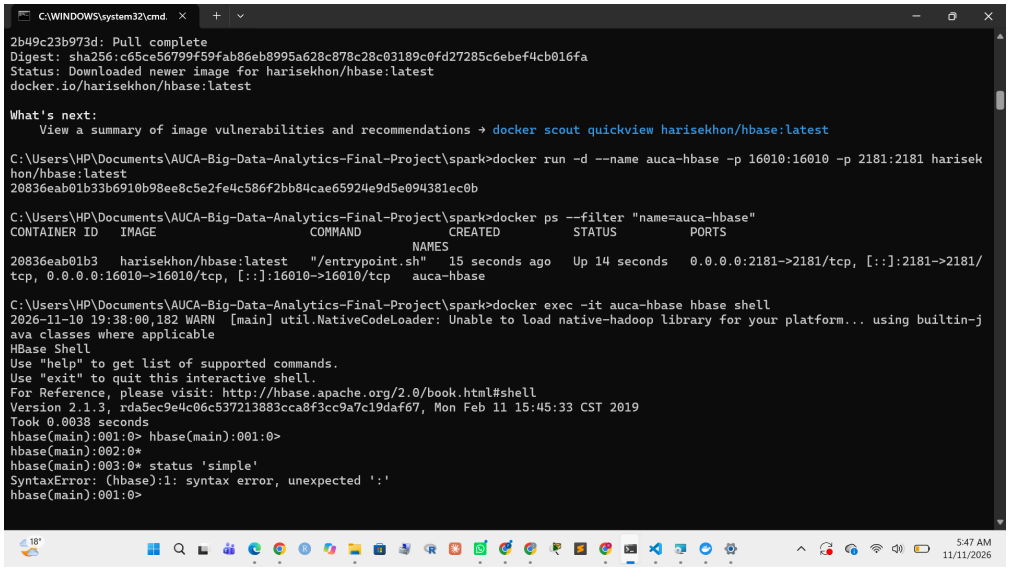




09 HBase container preparation

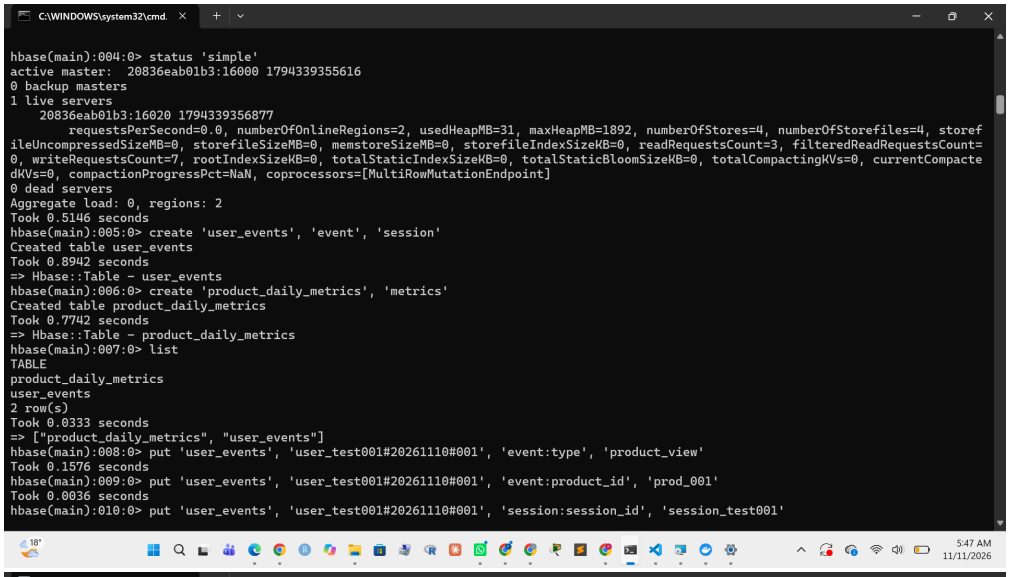
The HBase image was pulled and a container started. The HBase shell was opened; an initial shell command produced a syntax error.

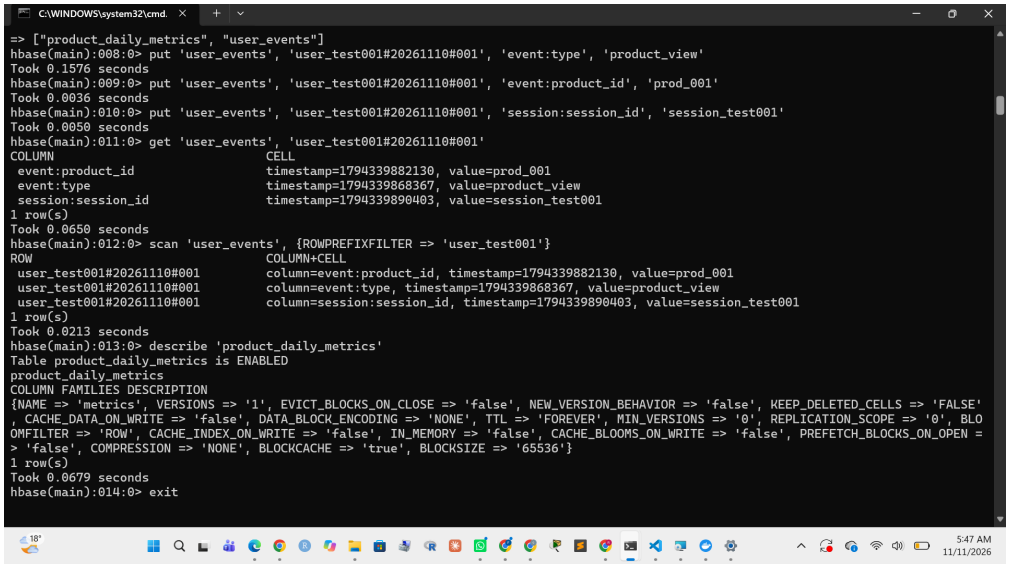




10 HBase schema and sample records

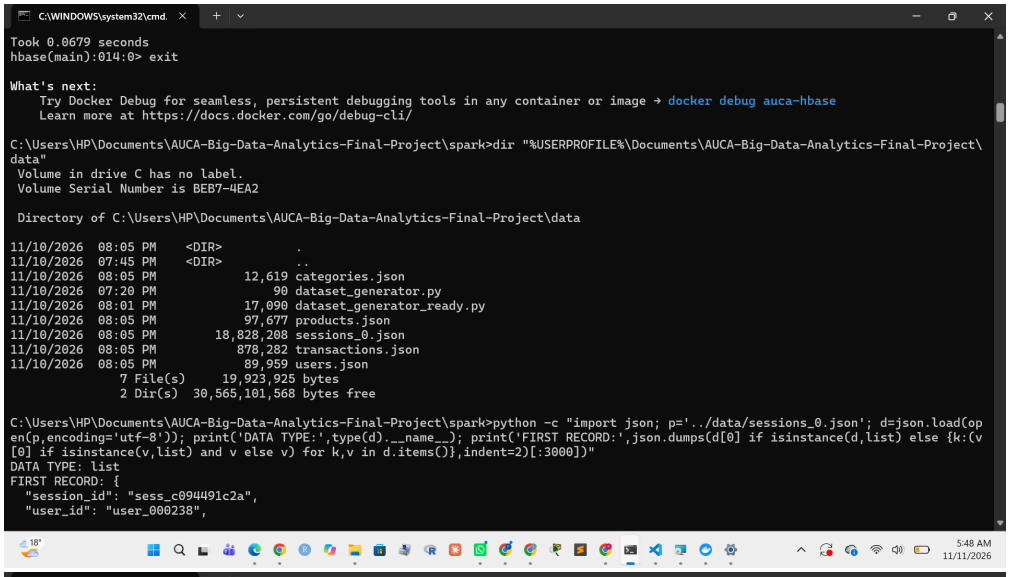
HBase status was checked. The user_events and product_daily_metrics tables were created, and sample put, get, scan and describe operations were executed.

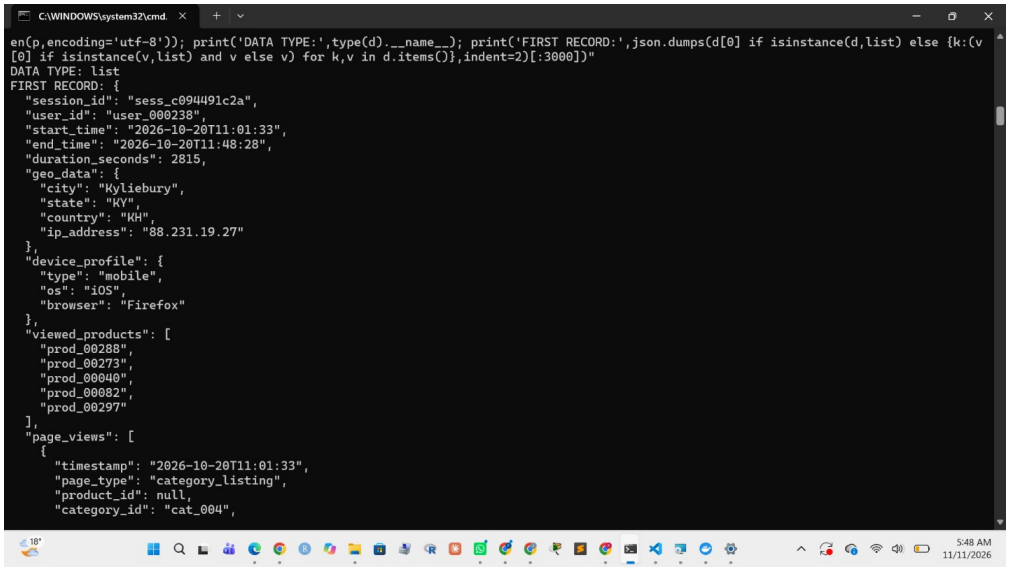




11 Generated data verification

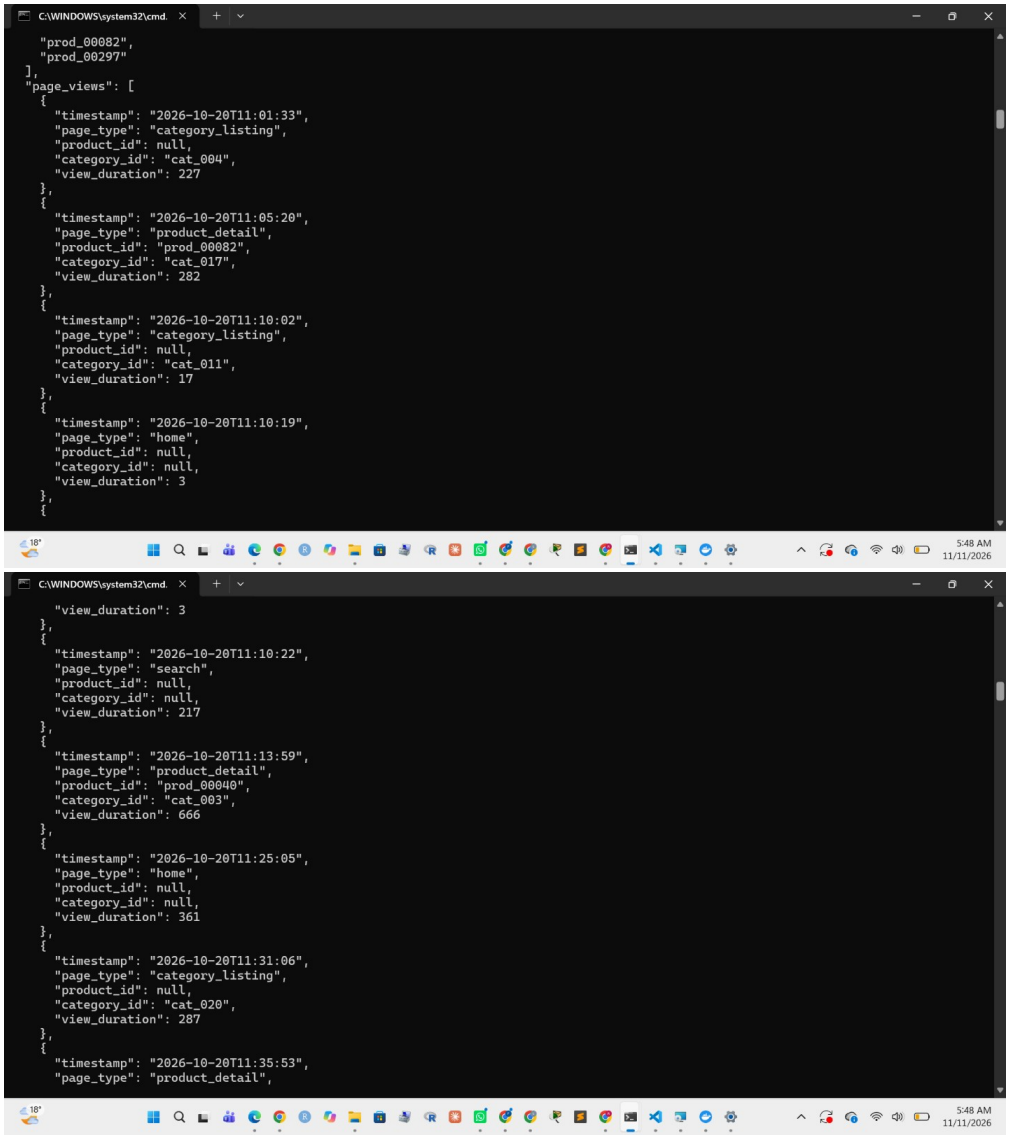
The local JSON dataset files were listed and a session record was inspected to confirm field structure before loading.





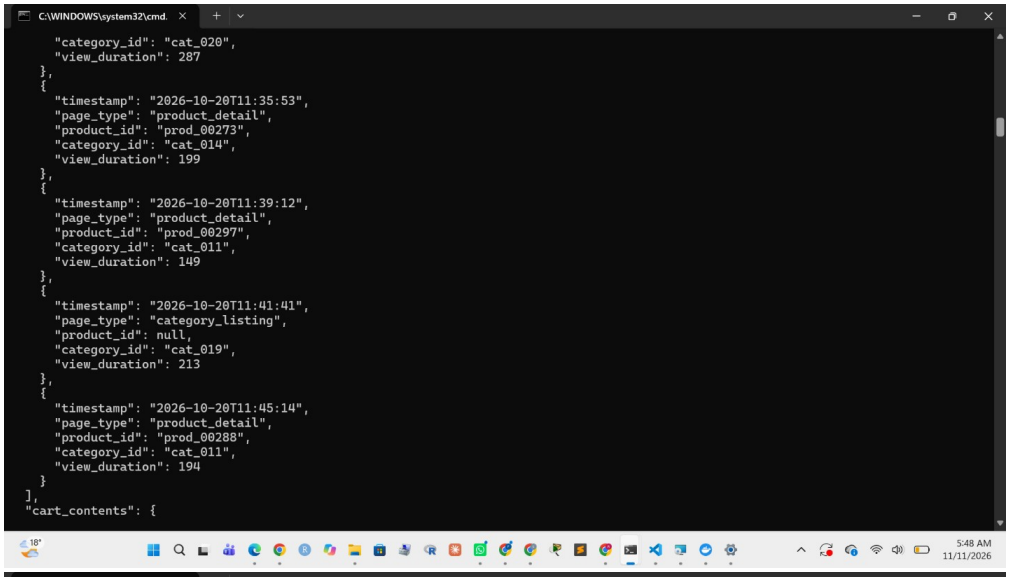
12 Session JSON inspection, continued

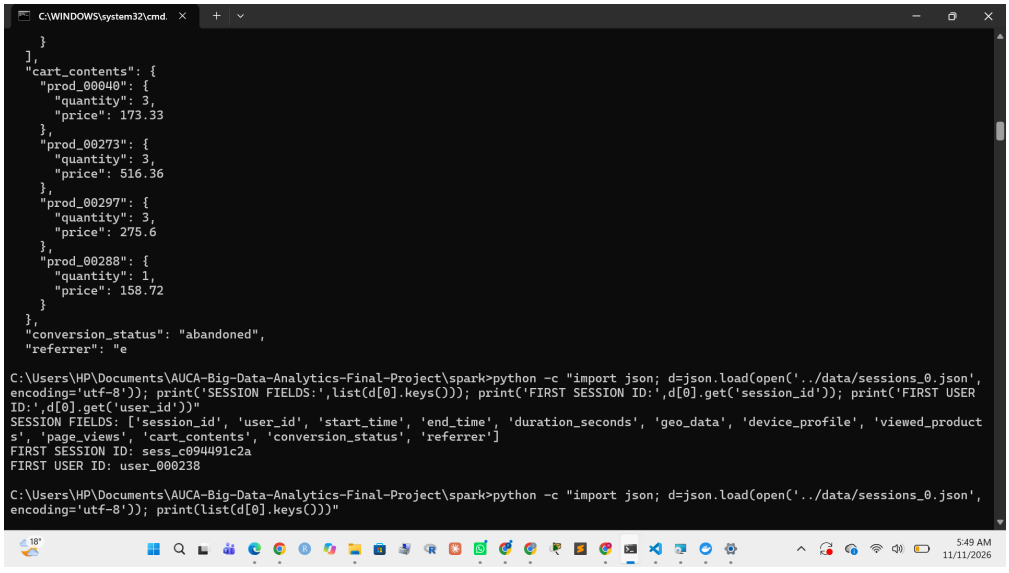
The session record contains a sequence of page-view events and associated product/category fields, supporting downstream behavioral analysis.



13 Session cart and status fields

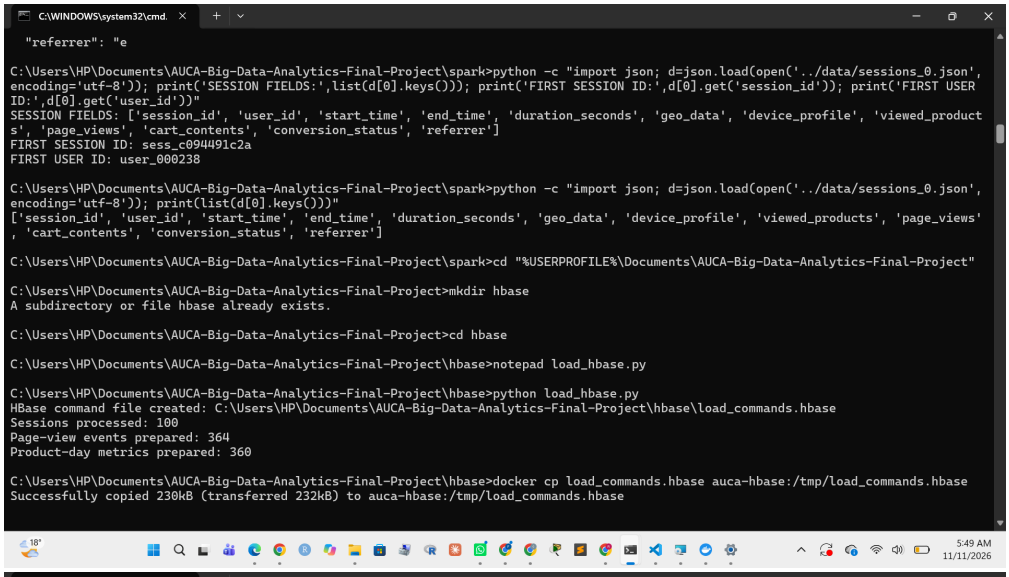
The sample session includes cart contents, quantities, prices and a conversion-status field. The session schema was checked before HBase loading.

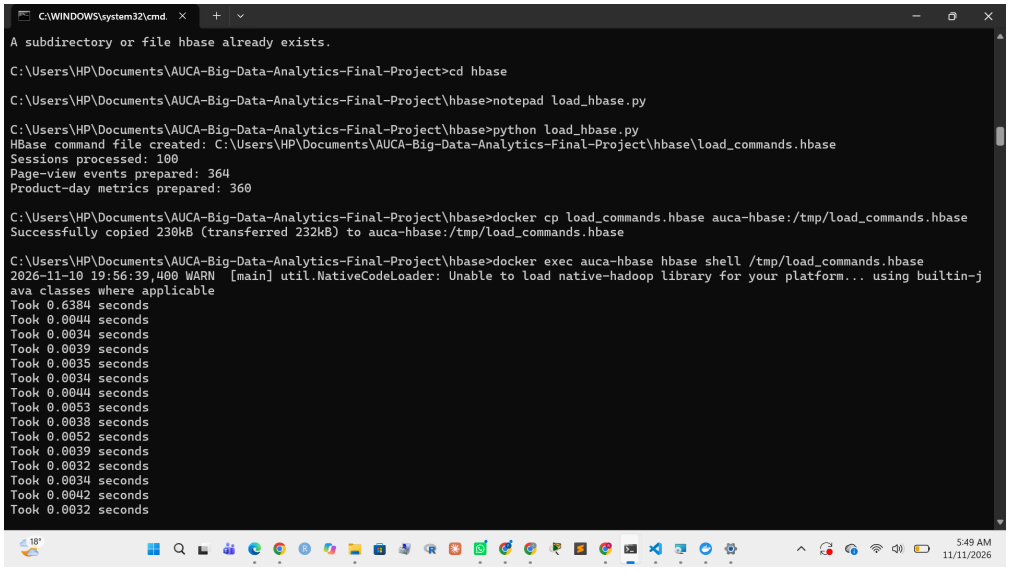




14 HBase loading pipeline

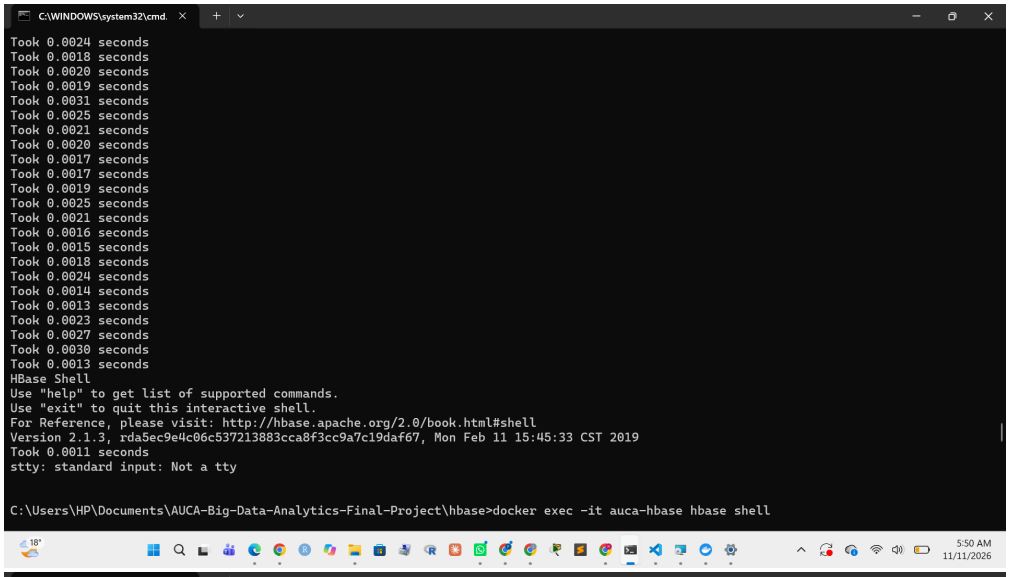
The HBase loader prepared page-view events and daily product metrics and transferred generated HBase commands into the running container.

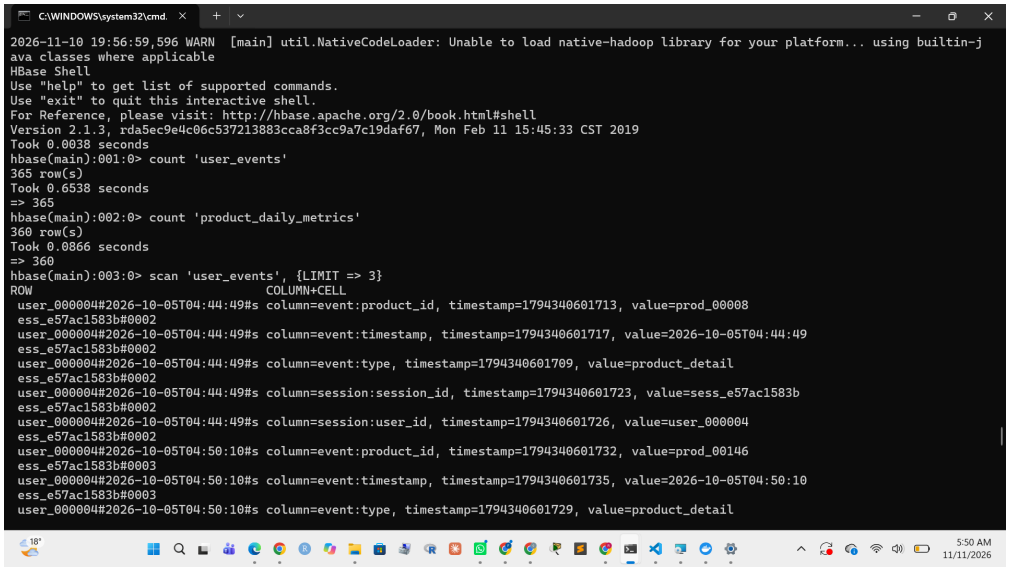




15 HBase row-count verification

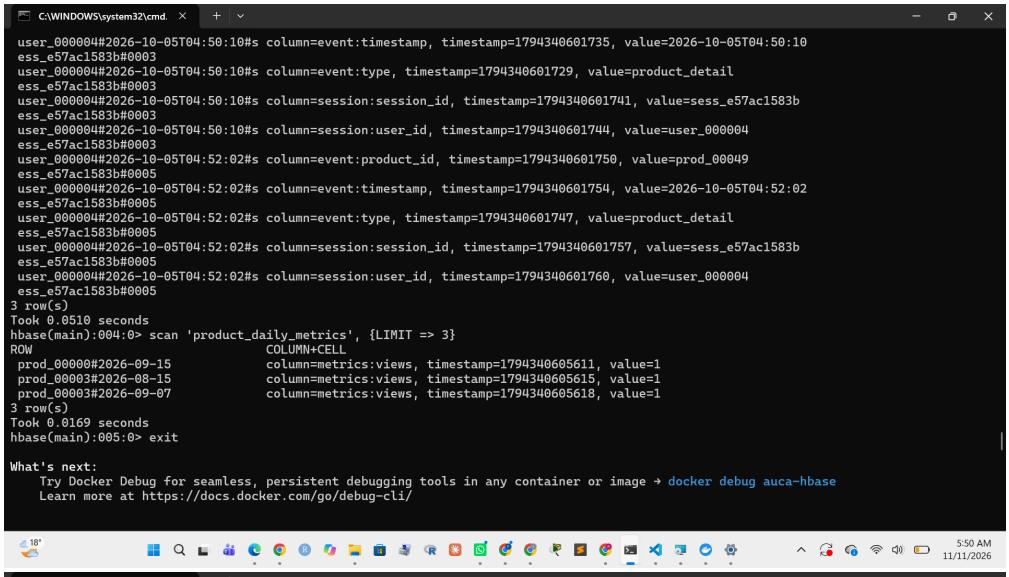
HBase count commands confirmed 365 rows in user_events and 360 rows in product_daily_metrics. A sample event scan was executed.

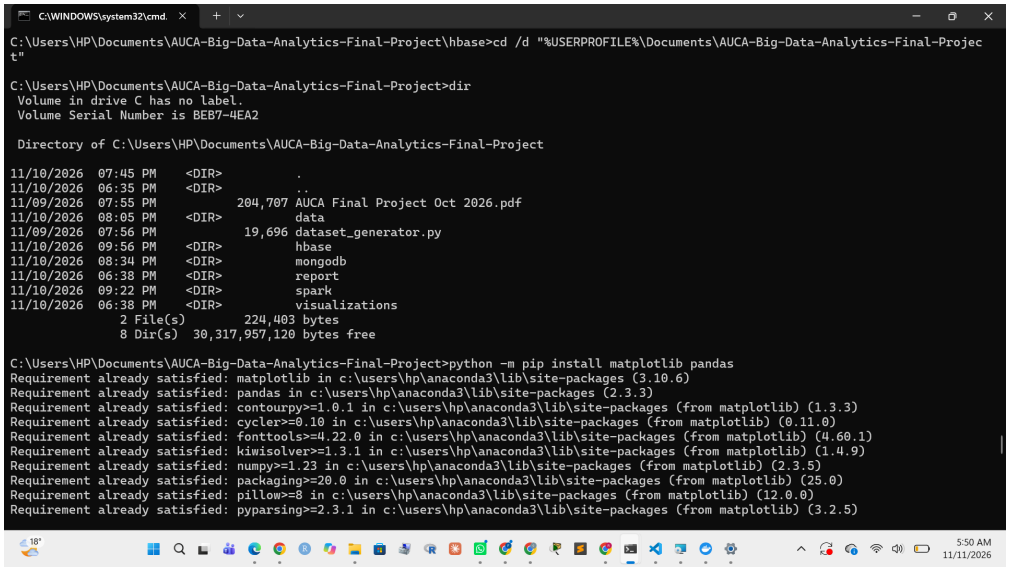




16 HBase scan verification and Python dependencies

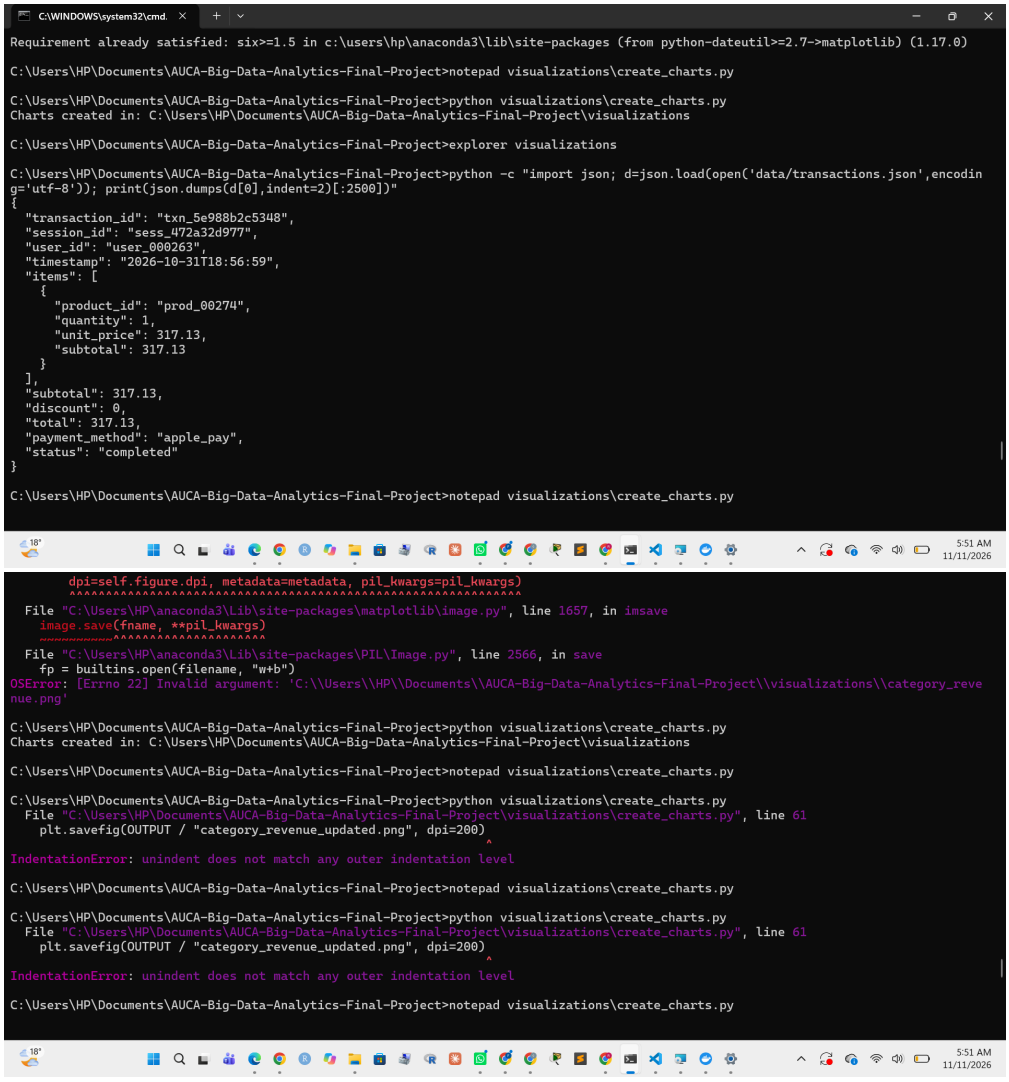
Sample rows from both HBase tables were scanned. The project directory and plotting dependencies were checked.





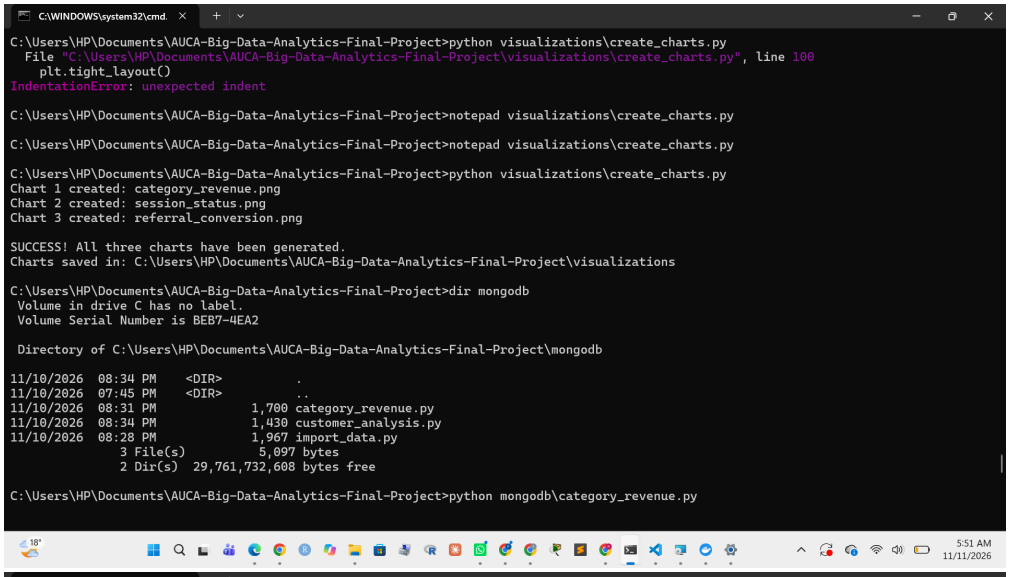
17 Visualization generation and debugging

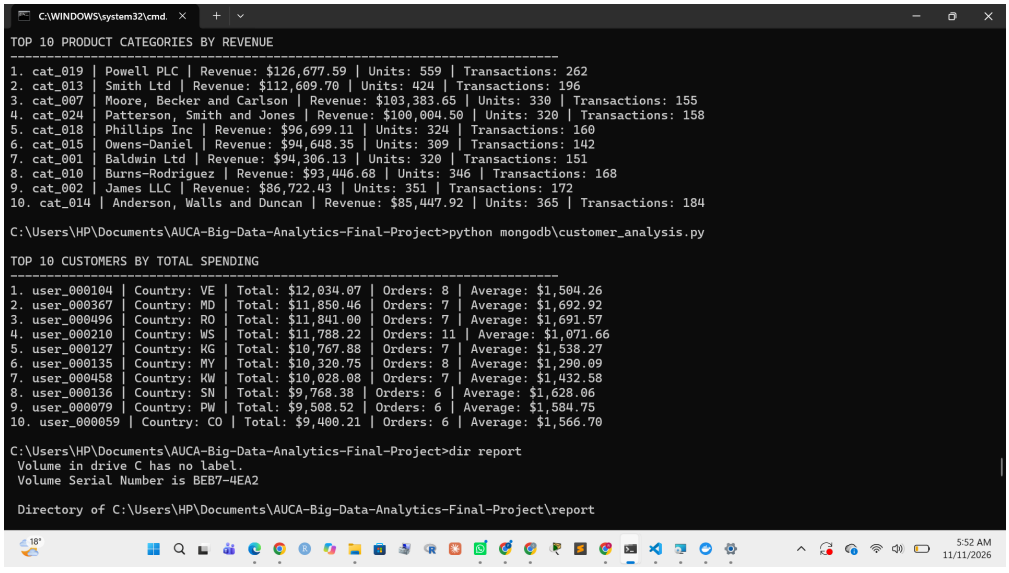
The visualization script was run. Initial plotting errors and indentation issues were encountered and investigated.



18 Successful chart generation and MongoDB results

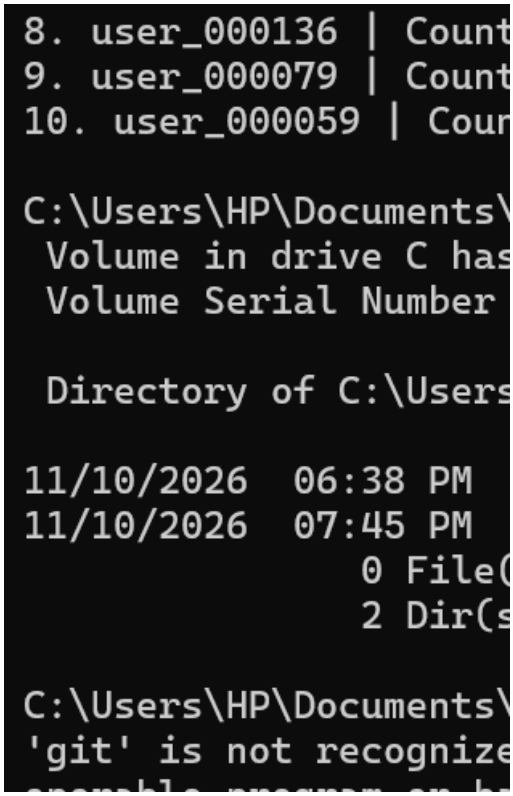
The corrected plotting script successfully generated category_revenue.png, session_status.png and referral_conversion.png. MongoDB queries were rerun.

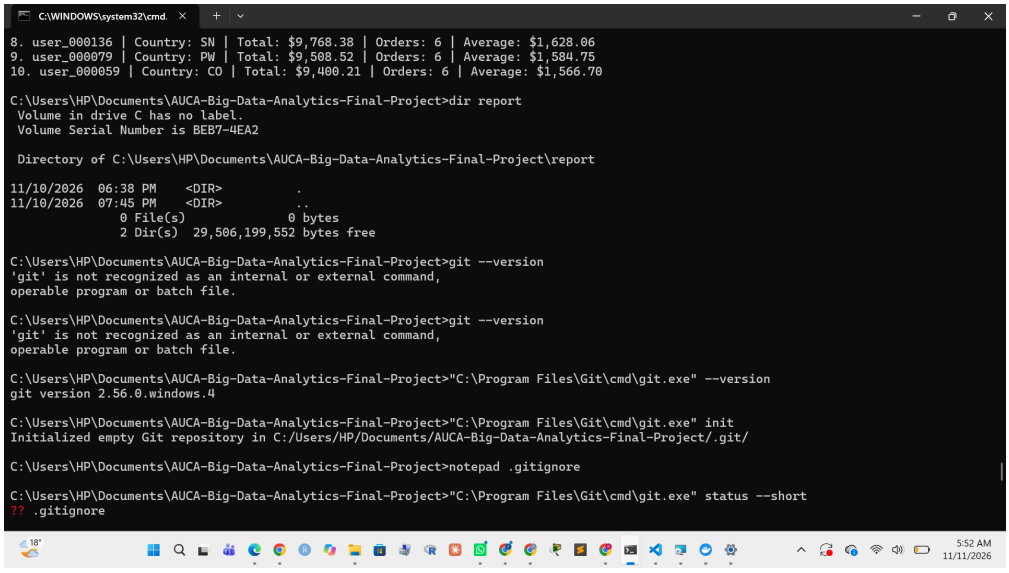




19 Git installation and repository setup

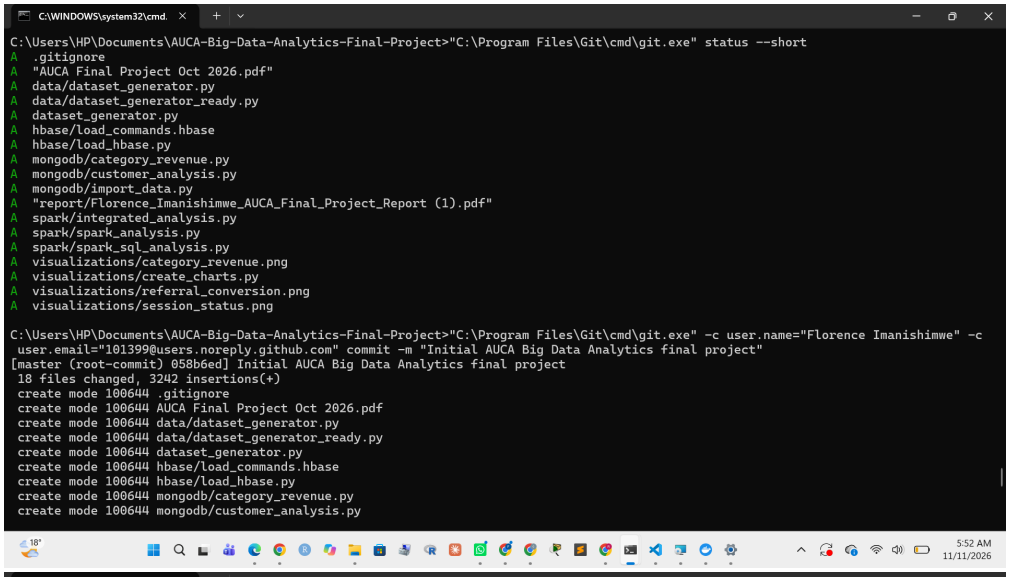
Git was not initially recognized on PATH, so its installed executable was called directly to initialize and inspect the repository.

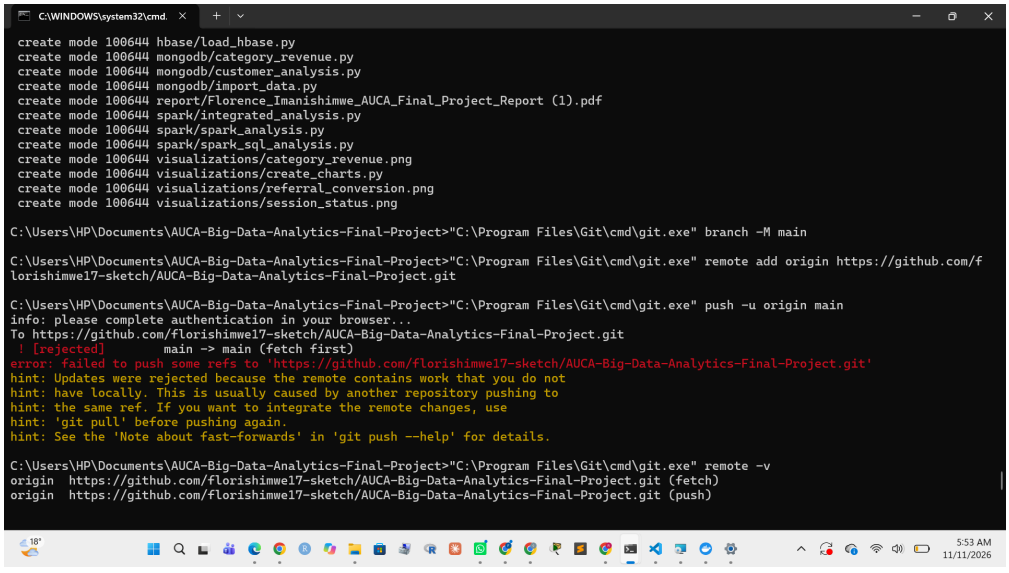




20 Git staging and initial commit

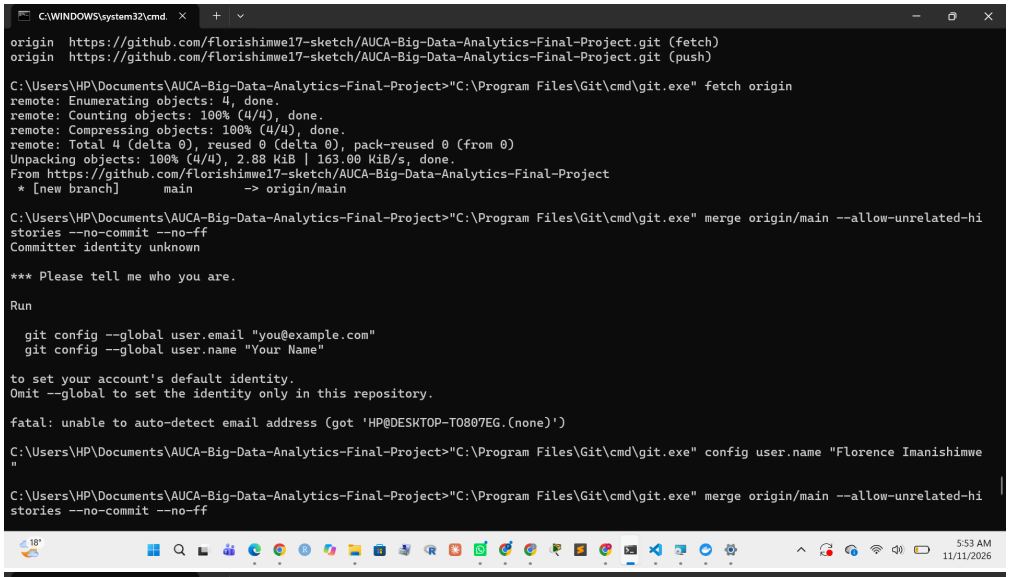
Project source files and charts were staged and committed. An initial push was rejected because the remote repository already contained changes.

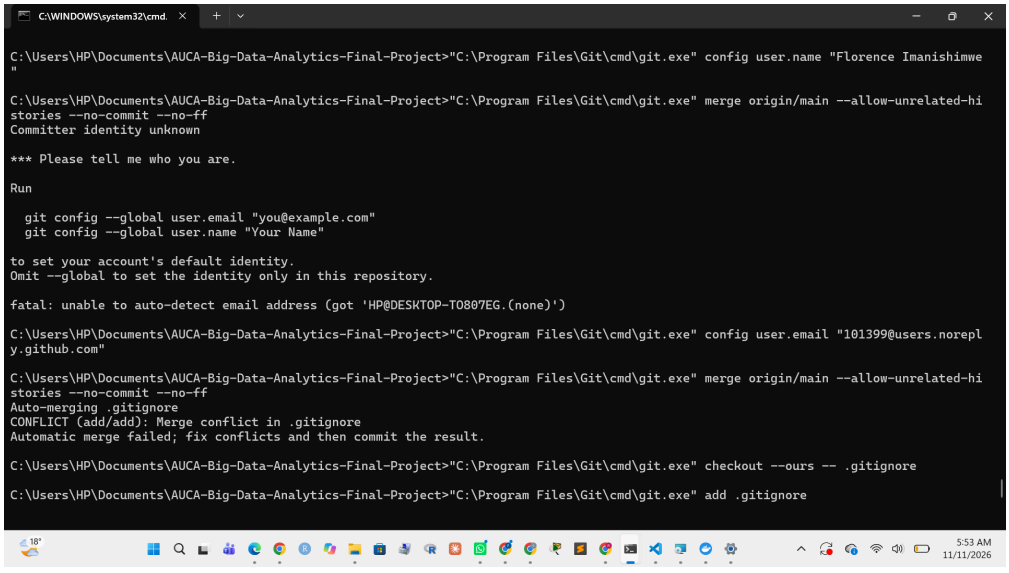




21 Git synchronization troubleshooting

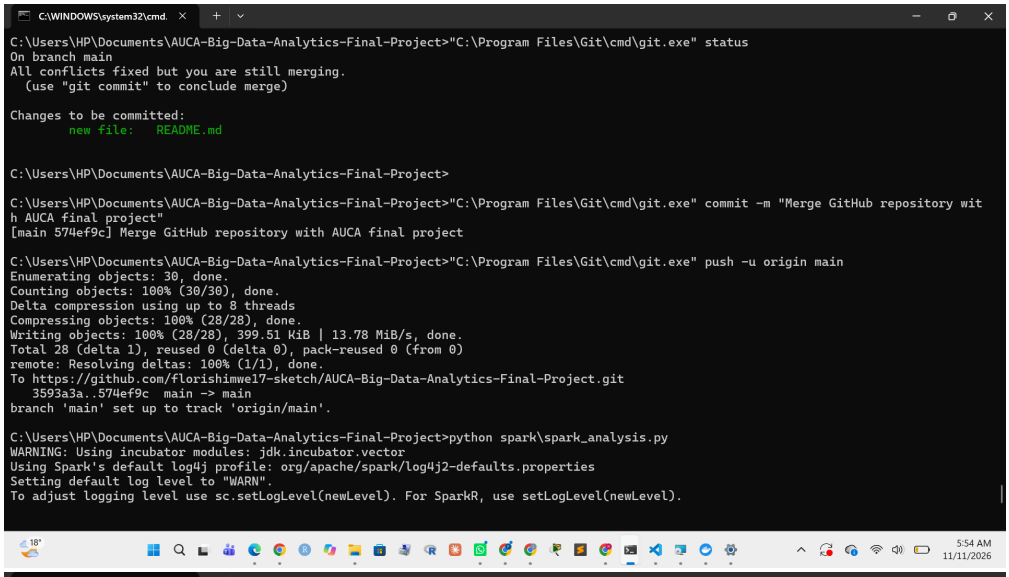
Remote changes were fetched and Git identity and merge conflicts were addressed before synchronization.

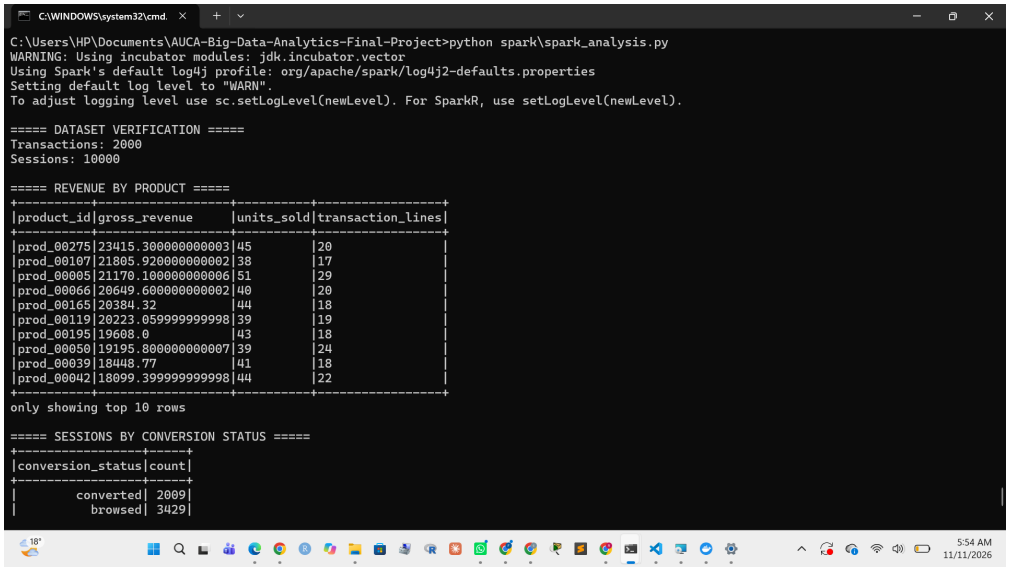




22 Successful Git push and Spark verification

The GitHub push succeeded. Spark analysis was rerun, showing dataset verification and revenue/session outputs.





23 Product affinity and final Git synchronization

Spark product-affinity analysis identified frequently co-purchased product pairs. A subsequent pull --rebase and push synchronized changes to GitHub.

